

Development of Large Systems

Example of exam topics and questions

1. Microservice design and service boundaries
 - a. Importance of service boundaries
 - b. From requirements to microservices
 - c. Data ownership and databases per service
 - d. Design trade-offs
 - e. Design decisions in your project
 - f. What would be the consequences of choosing *different* service boundaries?
2. Challenges and Advantages of Distributed Systems
 - a. challenges and trade-offs
 - b. solutions
 - c. Which of these did you deal with in your system?
 - d. Advantages in comparison to monolith architecture
 - e. When and why should we choose distributed system architecture?
 - f. Which advantage of distributed systems only appears at scale?
3. Idempotent API:
 - a. Implementation for HTTP retries and message processing.
 - b. Did you implement it in your project?
 - c. What are the trade-offs of implementing idempotency?
4. CQRS pattern:
 - a. In which scenarios should we use it?
 - b. How do we implement it?
 - c. Did you implement it in your system?
 - d. How does CQRS affect data consistency and transactions?
 - e. What problems can CQRS introduce if used incorrectly?
5. Immutable data / database
 - a. Snapshot pattern – explanation + implementation
 - b. Tombstone pattern – explanation + implementation
 - c. Did you implement it in your system?
 - d. Pros and Cons
 - e. What are the operational costs of immutable data?
 - f. When would mutable data be the better choice?
6. Database migrations
 - a. What is it?
 - b. Why should we use them?
 - c. Did you implement migrations in your system?
 - d. How do we deal with existing production data?

- e. Why are database migrations harder in distributed systems than monoliths?
- 7. Versioning:
 - a. Why do we need versioning?
 - b. What can we version? What is the benefit? How do we do it?
 - i. Code
 - ii. Data
 - iii. API
 - iv. Events
 - c. What happens if producers and consumers evolve at different speeds?
 - d. How does versioning relate to backward compatibility?
- 8. Event-driven communication and messaging:
 - a. Pros and cons compared to request-response communication
 - b. When should we use it?
 - c. What is the difference between asynchronous and synchronous communication?
 - d. Message Brokers / Queues,
 - e. Events / messages vs commands
 - f. At-least-once vs at-most-once message delivery
 - g. Message ordering guarantees
 - h. Dead letter queues
 - i. What new problems does event-driven architecture introduce?
 - j. When does event-driven communication become a disadvantage?
- 9. Commutative message handlers:
 - a. How can they be implemented?
 - b. How did you implement it in your project?
 - c. Why are commutative handlers useful in eventually consistent systems?
 - d. How do commutative handlers reduce the need for ordering guarantees?
- 10. Observability
 - a. Logging
 - b. Monitoring and metrics
 - c. Tracing
 - d. Which tools do you know?
 - e. Did you implement it in your project?
 - f. Why is observability more difficult in distributed systems?
- 11. Data consistency, Distributed Transactions
 - a. Why traditional ACID transactions don't scale across services.
 - b. Local vs distributed transactions.
 - c. Two-Phase Commit (2PC) - pros & cons.
 - d. Saga pattern vs distributed transactions.
 - e. What is eventual consistency.
 - f. How did you ensure data consistency in your system?
 - g. Why is strong consistency expensive in distributed systems?
- 12. Delivery & Operations of Distributed Systems

- a. What is a CI/CD pipeline and why is it important in distributed systems?
 - b. How does source control and branching support CI/CD?
 - i. Name different branching strategies
 - c. Deployment strategies
 - i. Blue-green deployment
 - ii. Canary releases
 - iii. Rolling updates
 - d. How do these reduce risk in production?
 - e. Why do deployment strategies matter more in distributed systems than monoliths?
13. Testing distributed systems
- a. Why testing is harder in distributed systems
 - b. What is a system (end-to-end) test in distributed system?
 - c. Difference between testing a single service and the full system
 - d. Why can a system work in isolation but fail when fully integrated?
 - e. What types of bugs are only found by system tests?
 - f. Why do system tests in distributed systems often produce non-deterministic results?
14. Scalability strategies
- a. Vertical vs horizontal scaling
 - b. Stateless vs stateful services
 - c. Sharding and partitioning
 - d. Auto-scaling triggers and metrics
 - e. How did your system scale?
 - f. Why does horizontal scaling often require stateless services?
15. Infrastructure for distributed systems
- a. Cloud Computing
 - b. Virtual machines vs Containers
 - c. Kubernetes
 - d. Serverless functions
 - e. Load balancing and scaling
 - f. IaC – Infrastructure as Code
 - g. Why is Infrastructure as Code important for reliability?
 - h. When does Kubernetes add more complexity than value?
16. Fault tolerance & resilience
- a. What does fault tolerance mean in distributed systems?
 - i. Why partial failure is unique to distributed systems
 - ii. Why “the system is not down, just parts of it”
 - iii. How this affects design decisions
 - b. Common failure types (network, partial failure, timeout)
 - c. Patterns:
 - i. Retry (with backoff)
 - ii. Circuit breaker
 - iii. Timeout

iv. Bulkhead

d. Which failures did you experience and how did you handle them?

17. Security in distributed systems

a. Common security challenges

b. Authentication vs authorization

c. Service-to-service security

d. How did you secure your system?